

本项目领域自适应数学原理教程

面向 seu_undergraduate_final_year_project 仓库

依据当前工作区代码与配置整理

目录

1	写在前面	2
2	问题定义与仓库中的数学对象	2
2.1	任务是什么	2
2.2	样本张量到底长什么样	3
2.3	统一模型骨架	3
2.4	统一监督项	3
2.5	统一的对齐权重调度	4
3	六个基线方法	4
3.1	Source Only: 只做源域监督	4
3.2	CORAL: 对齐二阶统计量	4
3.3	DAN: 用 MK-MMD 对齐边缘分布	5
3.4	DANN: 域对抗学习	5
3.5	CDAN: 把类别条件也送进对抗器	6
3.6	DeepJDOT: 联合特征距离与类别代价的最优传输	7
4	RCTA: 可靠性门控的类条件时序自适应	7
4.1	先看整体思路	7
4.2	教师网络与弱强双视图	8
4.3	可靠性评分: 四个分量一起投票	8
4.4	门控机制: 不是全局阈值, 而是按类筛 top 比例	9
4.5	基础对齐分支与 MCC	9
4.6	伪标签损失	10
4.7	原型记忆: 仓库里的版本比论文草稿更具体	10
4.8	一致性、半可靠样本和不可靠样本	11
4.9	课程化总目标	11
4.10	一步训练之后还会发生什么	12

1 写在前面	2
5 把七种方法放在一张图里理解	12
6 结语	12

1 写在前面

这份教程不是泛泛而谈的“领域自适应概论”，而是严格对照当前仓库里的实现来讲数学。它主要依据以下文件整理：

- 方法实现：src/methods/*.py
- 损失实现：src/losses/domain.py
- 共享骨干：src/backbones/fcn.py
- 默认方法配置：configs/method/*.yaml
- 当前主线实验配置：configs/experiment/*.yaml

因此，下文遇到“标准论文公式”和“仓库当前工程实现”不完全一致时，一律以仓库代码为准。

2 问题定义与仓库中的数学对象

2.1 任务是什么

本项目处理的是无监督领域自适应故障诊断。设源域训练集为

$$\mathcal{D}_s = \{(x_i^s, y_i^s)\}_{i=1}^{n_s}, \quad (1)$$

目标域训练集为

$$\mathcal{D}_t = \{x_j^t\}_{j=1}^{n_t}, \quad (2)$$

其中目标域训练阶段没有标签。

在仓库当前数据设定中，不同 TEP 操作模式就是不同领域：

$$\mathcal{M} = \{\text{mode1}, \text{mode2}, \text{mode3}, \text{mode4}, \text{mode5}, \text{mode6}\}. \quad (3)$$

标签空间固定为 29 类：

$$\mathcal{Y} = \{0, 1, \dots, 28\}. \quad (4)$$

2.2 样本张量到底长什么样

根据当前 `data/benchmark/manifest.json`, 6 个域都共享相同单样本长度:

$$x \in \mathbb{R}^{600 \times 34}. \quad (5)$$

仓库在送入网络前会转为 channels-first, 因此真正进入模型的是

$$x' \in \mathbb{R}^{34 \times 600}. \quad (6)$$

这里 34 是过程变量通道数, 600 是时间窗口长度。当前实现对每个域单独做标准化, 随后再划分训练/评估索引, 这一点由 `configs/data/te_da.yaml` 与 `src/datasets/te_torch_dataset.py` 共同决定。

2.3 统一模型骨架

无论是 Source Only、CORAL、DAN、DANN、CDAN、DeepJDOT 还是 RCTA, 仓库都共享同一个

$$f(x) = h(\phi(x)) \quad (7)$$

结构:

- 编码器 ϕ : 1D FCN, 输出 128 维特征;
- 分类头 h : 隐藏层维度默认 128, 输出 29 类 logits。

对应到 `src/backbones/fcn.py`, 编码器是三层卷积:

$$z^{(1)} = \text{ReLU}(\text{Norm}(\text{Conv1D}_{9,128}(x'))), \quad (8)$$

$$z^{(2)} = \text{ReLU}(\text{Norm}(\text{Conv1D}_{5,256}(z^{(1)}))), \quad (9)$$

$$z^{(3)} = \text{ReLU}(\text{Norm}(\text{Conv1D}_{3,128}(z^{(2)}))), \quad (10)$$

再沿时间维做全局平均池化:

$$f = \phi(x') = \frac{1}{T} \sum_{t=1}^T z_{:,t}^{(3)} \in \mathbb{R}^{128}. \quad (11)$$

分类头输出 logits:

$$z = h(f) \in \mathbb{R}^{29}, \quad p = \text{softmax}(z). \quad (12)$$

2.4 统一监督项

所有方法都保留源域监督分类项。对一个源域小批量 $\mathcal{B}_s$, 交叉熵写成

$$\mathcal{L}_{cls}^s = -\frac{1}{|\mathcal{B}_s|} \sum_{(x_i^s, y_i^s) \in \mathcal{B}_s} \log p_{i, y_i^s}^s. \quad (13)$$

需要注意的是，训练器支持多源场景，但当前做法不是“多源专用网络”，而是把多个源 loader 取出的 batch 直接拼起来：

$$\mathcal{B}_s = \bigcup_{m=1}^M \mathcal{B}_{s_m}. \quad (14)$$

这个行为来自 `SingleSourceMethodBase.merge_source_batches()`。所以从数学上讲，多源只是把监督样本池扩大了；方法本体仍按单一共享特征空间建模。

2.5 统一的对齐权重调度

多个方法都会用到对齐权重调度器 `AdaptationWeightScheduler`。若基础权重为 λ_0 ，训练步数为 t ，最大调度步数为 T ，则 warm-start 形式为

$$\lambda(t) = \lambda_0 \left(\frac{2}{1 + \exp(-\alpha \min(t/T, 1))} - 1 \right). \quad (15)$$

此外代码还支持 constant 与 linear 两种形式：

$$\lambda_{\text{constant}}(t) = \lambda_0, \quad \lambda_{\text{linear}}(t) = \lambda_0 \min(t/T, 1). \quad (16)$$

3 六个基线方法

3.1 Source Only: 只做源域监督

这是最基础的参照组。它完全忽略目标域训练样本，只最小化

$$\mathcal{L}_{\text{SO}} = \mathcal{L}_{\text{cls}}^s. \quad (17)$$

它的重要性不在于“强”，而在于回答一个最根本的问题：如果一个领域自适应方法最终还不如 Source Only，那说明它引入的对齐信号并没有真正缓解跨工况分布偏移。

3.2 CORAL: 对齐二阶统计量

CORAL 的思想是：如果源域与目标域特征协方差接近，那么两域的整体几何形状会更接近。设源域特征矩阵与目标域特征矩阵分别为

$$F_s \in \mathbb{R}^{n_s \times d}, \quad F_t \in \mathbb{R}^{n_t \times d}. \quad (18)$$

其均值为

$$\mu_s = \frac{1}{n_s} \sum_i f_i^s, \quad \mu_t = \frac{1}{n_t} \sum_j f_j^t, \quad (19)$$

协方差为

$$C(F) = \frac{(F - \bar{F})^\top (F - \bar{F})}{\max(n - 1, 1)}. \quad (20)$$

当前仓库实现的 CORAL 损失是

$$\mathcal{L}_{coral} = \frac{\|C(F_s) - C(F_t)\|_F^2}{4d^2} + \mathbf{1}_{align_mean} \cdot \frac{\|\mu_s - \mu_t\|_2^2}{d}. \quad (21)$$

于是总目标为

$$\mathcal{L}_{CORAL} = \mathcal{L}_{cls}^s + \lambda(t)\mathcal{L}_{coral}. \quad (22)$$

如何理解 协方差控制“云团形状”，均值项控制“云团中心”。仓库默认方法配置 `configs/method/coral.yaml` 把 `align_mean` 设为 `false`；但某些实验配置会在 `method_overrides` 里显式把它打开。

3.3 DAN：用 MK-MMD 对齐边缘分布

DAN 通过多核最大均值差异（MK-MMD）逼近

$$P_s(f) \approx P_t(f). \quad (23)$$

给定核函数族 $\{k_u\}_{u=1}^U$ ，高斯核常写成

$$k_u(a, b) = \exp\left(-\frac{\|a - b\|_2^2}{2\sigma_u^2}\right). \quad (24)$$

MMD 的经典形式是

$$\text{MMD}^2(F_s, F_t) = \left\| \frac{1}{n_s} \sum_i \varphi(f_i^s) - \frac{1}{n_t} \sum_j \varphi(f_j^t) \right\|_{\mathcal{H}}^2. \quad (25)$$

当前仓库把多个高斯核相加，并通过一个 `index matrix` 构造线性或非线性估计器，因此总损失可概括为

$$\mathcal{L}_{DAN} = \mathcal{L}_{cls}^s + \lambda(t)\mathcal{L}_{mkmmmd}. \quad (26)$$

如何理解 如果说 CORAL 是对齐二阶统计量，那么 DAN 是在 RKHS 里对齐整体现象分布。仓库默认配置里 `linear_mmd=true`，而主线 `benchmark` 配置里会把它改为 `false`，使用更完整的二次型估计。

3.4 DANN：域对抗学习

DANN 的核心是让特征既要有分类性，又要让域判别器分不清来自源域还是目标域。设域判别器为 $D(\cdot)$ ，定义源域域标签为 1，目标域域标签为 0。当前代码里的域对抗损失写成

$$\mathcal{L}_{adv} = \frac{1}{2} \left(\text{BCE}(D(\text{GRL}(F_s)), 1) + \text{BCE}(D(\text{GRL}(F_t)), 0) \right). \quad (27)$$

总目标为

$$\mathcal{L}_{DANN} = \mathcal{L}_{cls}^s + \lambda(t)\mathcal{L}_{adv}. \quad (28)$$

这里关键不是公式本身，而是梯度反转层 GRL。它在前向传播时相当于恒等映射，但反向传播时把梯度乘以负系数：

$$\frac{\partial \mathcal{L}}{\partial f} \leftarrow -\lambda_{\text{grl}} \frac{\partial \mathcal{L}}{\partial f}. \quad (29)$$

这样域判别器在努力分域，编码器却被迫学出更域不敏感的特征。

3.5 CDAN：把类别条件也送进对抗器

CDAN 认为只对齐边缘分布不够，因为多类别任务里更关键的是“同类跨域靠近、异类跨域不要被混在一起”。因此它不是只把特征 f 送进域判别器，而是构造条件表示

$$g = f \otimes p, \quad (30)$$

其中 $p = \text{softmax}(z)$ 是类别概率。

当前仓库支持两种条件映射：

1. 确定性多线性映射：

$$g = \text{vec}(pf^\top) \in \mathbb{R}^{Kd}, \quad (31)$$

2. 随机映射：

$$g = \frac{(fR_f) \odot (pR_p)}{\sqrt{d_r}}. \quad (32)$$

当前实现里， p 在进入条件映射前会被 detach，也就是 stop-gradient。这意味着对抗项主要通过特征分支起作用，而不会把域判别梯度直接回传到 softmax 概率本身。

若记条件域判别器为 D_c ，则对抗项可写为

$$\mathcal{L}_{cadv} = \text{BCE}(D_c(\text{GRL}(g_s)), 1) + \text{BCE}(D_c(\text{GRL}(g_t)), 0). \quad (33)$$

CDAN 还支持熵加权：对每个样本给权重

$$w_i \propto 1 + \exp(-H(p_i)), \quad (34)$$

熵越低，权重越大。

此外，仓库还实现了可选的 MCC 正则。设目标域 logits 为 z_t ，温度为 τ ，则

$$Q = \text{softmax}(z_t/\tau). \quad (35)$$

MCC 会构造类别混淆矩阵并惩罚非对角部分，总体可理解为

$$\mathcal{L}_{mcc} \uparrow \iff \text{目标域类别边界更混乱}. \quad (36)$$

因此，CDAN 的总目标是

$$\mathcal{L}_{\text{CDAN}} = \mathcal{L}_{cls}^s + \lambda(t)\mathcal{L}_{cadv} + \lambda_{mcc}\mathcal{L}_{mcc}. \quad (37)$$

3.6 DeepJDOT: 联合特征距离与类别代价的最优传输

DeepJDOT 不是靠“一个判别器”来对齐, 而是显式建立源域样本和目标域样本之间的运输计划。设 batch 内源域特征为 $\{f_i^s\}_{i=1}^B$, 目标域特征为 $\{f_j^t\}_{j=1}^B$, 目标域 logits 为 $\{z_j^t\}_{j=1}^B$, 当前实现中的代价矩阵为

$$M_{ij} = \lambda_{dist} \frac{\|f_i^s - f_j^t\|_2^2}{d} + \lambda_{cl} (-\log \text{softmax}(z_j^t)_{y_i^s}). \quad (38)$$

然后求解

$$\mathcal{L}_{ot} = \min_{\gamma \in \Pi(a,b)} \langle \gamma, M \rangle, \quad (39)$$

其中 a, b 是均匀边缘分布, $\Pi(a, b)$ 表示满足行列约束的运输计划集合。

因此总目标为

$$\mathcal{L}_{\text{DeepJDOT}} = \mathcal{L}_{cls}^s + \lambda(t) \mathcal{L}_{ot}. \quad (40)$$

如何理解 DANN 和 CDAN 都是在“分域/混域”的意义上对齐; DeepJDOT 则更像“显式匹配谁该和谁靠近”。在这个仓库里, OT 求解器依赖 POT 包; 普通 DeepJDOT 方法可以在 Sinkhorn 与 EMD 间切换。

4 RCTA: 可靠性门控的类条件时序自适应

4.1 先看整体思路

RCTA 不是简单把几个损失加起来, 而是把四件事串成闭环:

1. 用 EMA 教师在目标域弱增强视图上产生更稳的软预测;
2. 用可靠性评分决定哪些目标样本值得信;
3. 用原型记忆把类结构显式拉出来;
4. 用课程调度控制这些信号什么时候开始发力、发多大力。

更重要的是, 当前仓库中的 RCTA 不是单一固定版本。它支持三种基础对齐分支:

$$\text{base_align} \in \{\text{cdan}, \text{dann}, \text{deepjdot}\}. \quad (41)$$

默认方法配置 `configs/method/rcta.yaml` 使用的是 `cdan`, 而不是论文草稿里曾经写过的固定 DANN 底座; 与此同时, 一些 experiment config 又会通过 `method_overrides` 把它切成 `dann`。

4.2 教师网络与弱强双视图

对目标样本 x_j^t , 先构造

$$\tilde{x}_{j,w}^t = \mathcal{A}_w(x_j^t), \quad \tilde{x}_{j,s}^t = \mathcal{A}_s(x_j^t). \quad (42)$$

其中弱增强主要是小幅 jitter 和 scaling; 强增强还会加入时间遮挡与通道 dropout。也就是说, RCTA 的“Temporal”不是 RNN 意义上的时间建模, 而是围绕固定长度工业时序窗口做视图增强和时间掩蔽。

教师网络参数通过 EMA 更新:

$$\theta_{tea} \leftarrow \mu\theta_{tea} + (1 - \mu)\theta_{stu}. \quad (43)$$

教师在弱增强视图上输出

$$p_j^{tea} = \text{softmax}\left(\frac{z_j^{tea}}{T_{tea}}\right), \quad \hat{y}_j = \arg \max_k p_{jk}^{tea}. \quad (44)$$

这里 T_{tea} 是教师温度, 当前默认配置里是 1.4。

4.3 可靠性评分: 四个分量一起投票

当前代码给每个目标样本构造四个分量:

$$q_j = \max_k p_{jk}^{tea}, \quad (45)$$

$$e_j = 1 - \frac{-\sum_k p_{jk}^{tea} \log p_{jk}^{tea}}{\log K}, \quad (46)$$

$$u_j = p_{j,\hat{y}_j}^{stu}, \quad (47)$$

$$s_j = \begin{cases} \frac{1 + \cos(f_j^{tea}, c_{\hat{y}_j}^{ref})}{2}, & \hat{y}_j \text{ 对应参考原型已激活,} \\ 0.5, & \text{否则.} \end{cases} \quad (48)$$

它们分别对应:

- q_j : 教师最高置信度;
- e_j : 逆归一化熵, 越大越尖锐;
- u_j : 学生在强增强视图上对教师伪标签的响应;
- s_j : 教师特征与参考原型的相似度。

若原始权重为 $\omega_1, \omega_2, \omega_3, \omega_4$, 代码会先把它们归一化, 再得到

$$r_j = \text{clip}_{[0,1]}(\bar{\omega}_1 q_j + \bar{\omega}_2 e_j + \bar{\omega}_3 u_j + \bar{\omega}_4 s_j). \quad (49)$$

若 `pseudo_confidence_power` 不是 1, 还会再做幂次放缩:

$$r_j \leftarrow \text{clip}_{[0,1]}(r_j^\gamma). \quad (50)$$

为什么这四项要同时存在 只看最大概率，容易相信“自信但错”的样本；只看熵，容易忽略类结构；只看原型相似度，早期原型又不稳；只看强增强一致性，可能把过拟合的一致性也当成可靠。四个分量一起用，等于从概率、结构和视图一致性三条线交叉验真。

4.4 门控机制：不是全局阈值，而是按类筛 top 比例

门控分两步。

第一步：分数下界 当前步数为 t 时，门控阈值线性调度为

$$\tau(t) = \tau_{start} + (\tau_{end} - \tau_{start}) \min(t/T_\tau, 1). \quad (51)$$

第二步：按类取前若干比例 接受比例同样做课程调度：

$$\rho(t) = \rho_{start} + (\rho_{end} - \rho_{start}) \min(t/T_\rho, 1). \quad (52)$$

对每个预测类别 k ，先定义候选集合

$$I_k(t) = \{j \mid \hat{y}_j = k, r_j \geq \tau(t)\}. \quad (53)$$

然后只保留该类内部得分最高的

$$\lceil \rho(t) \cdot |I_k(t)| \rceil \quad (54)$$

个样本进入门控集合 Ω_t 。

为什么要按类筛 如果用全局 top 比例，头部类别会吃掉绝大部分伪标签预算。当前 29 类工业诊断任务里，按类预算能明显减少类别失衡对自训练的污染。

4.5 基础对齐分支与 MCC

RCTA 并不是一开始就做对齐。若当前步数 $t < t_{align}$ ，则

$$\mathcal{L}_{align} = 0. \quad (55)$$

只有到 `alignment_start_step` 之后，对齐分支才开始工作。

另外，若 `alignment_use_reliable_only=true` 且当前门控集合非空，那么参与对齐的目标域样本不是整批目标样本，而是

$$\{(\tilde{x}_{j,w}^t, \hat{y}_j) \mid j \in \Omega_t\}. \quad (56)$$

所以 RCTA 的对齐是“可靠样本优先”的，而不是把全部目标样本一股脑送进对齐器。

于是，

$$\mathcal{L}_{align} \in \{\mathcal{L}_{CDAN-align}, \mathcal{L}_{DANN-align}, \mathcal{L}_{DeepJDOT-align}\}. \quad (57)$$

如果启用 MCC，代码把它作用在“实际参与对齐的目标 logits”上，而不是永远对整批目标 logits 计算：

$$\mathcal{L}_{mcc} = \text{MCC}(z_{t,align}). \quad (58)$$

4.6 伪标签损失

对门控集合中的目标样本，学生在强增强视图上接受伪标签监督。若启用可靠性加权，当前实现是

$$\mathcal{L}_{pl} = \frac{\sum_{j \in \Omega_t} r_j \text{CE}(z_j^{stu}, \hat{y}_j)}{\sum_{j \in \Omega_t} r_j + \varepsilon}. \quad (59)$$

若关闭可靠性加权，则退化为普通均值交叉熵。

4.7 原型记忆：仓库里的版本比论文草稿更具体

对源域当前 batch，每个类别的 batch 原型是

$$\tilde{c}_k^{s, batch} = \text{Norm} \left(\frac{1}{|\mathcal{B}_{s,k}|} \sum_{i: y_i^s = k} f_i^s \right). \quad (60)$$

仓库同时维护两个跨步记忆库：

$$\{c_k^s\}_{k=1}^K, \quad \{c_k^t\}_{k=1}^K. \quad (61)$$

更新方式是动量平均：

$$c_k \leftarrow \text{Norm}(\beta c_k + (1 - \beta) \tilde{c}_k). \quad (62)$$

但当前实现里的参考原型 c_k^{ref} 不是简单三项线性组合，而是下面这个优先级逻辑：

$$c_k^{ref} = \begin{cases} \text{Norm}(c_k^s + c_k^t), & c_k^s, c_k^t \text{ 都已激活,} \\ c_k^s, & \text{仅源记忆激活,} \\ c_k^t, & \text{仅目标记忆激活,} \\ \tilde{c}_k^{s, batch}, & \text{仅当前源 batch 有该类,} \\ \text{inactive}, & \text{否则.} \end{cases} \quad (63)$$

也就是说，源记忆优先于当前源 batch；若目标记忆也存在，再与源记忆归一化融合。

基于此，原型吸引项定义为

$$\mathcal{L}_{att}(F, Y) = \frac{1}{|F|} \sum_i (1 - \cos(f_i, c_{y_i}^{ref})). \quad (64)$$

仓库里的 RCTA 原型损失不是单一一项，而是

$$\mathcal{L}_{proto} = \mathcal{L}_{att}^{src} + \mathcal{L}_{att}^{tgt, gate} + \frac{1}{2} \mathcal{L}_{soft_tgt} + \lambda_{sep} \mathcal{L}_{sep}. \quad (65)$$

其中，软目标原型项会对整批目标弱增强特征做软加权：

$$w_j^{soft} = \begin{cases} 1, & r_j \geq r_{rel}, \\ \frac{r_j - r_{semi}}{r_{rel} - r_{semi}}, & r_{semi} \leq r_j < r_{rel}, \\ 0, & r_j < r_{semi}, \end{cases} \quad (66)$$

$$\mathcal{L}_{soft_tgt} = \frac{\sum_j w_j^{soft} (1 - \cos(f_j^{t,w}, c_{y_j}^{ref}))}{\sum_j w_j^{soft} + \varepsilon}. \quad (67)$$

原型分离项则惩罚激活类别原型之间过高的相似度:

$$\mathcal{L}_{sep} = \text{mean}_{k < l} \left[\max(0, \cos(c_k^{ref}, c_l^{ref}) - m) \right]. \quad (68)$$

一个很关键的实现细节 当前代码中, 目标原型记忆库的更新使用的是所有目标 teacher 特征和 argmax 伪标签, 而不是只用门控通过的目标样本。所以“显式监督损失”与“原型记忆更新”采用的目标样本范围并不完全相同。

4.8 一致性、半可靠样本和不可靠样本

设学生在强增强视图上的概率为 p_j^{stu} , 教师概率为 p_j^{tea} 。当前一致性不是 KL 散度, 而是逐样本均方差:

$$v_j = \frac{1}{K} \|p_j^{stu} - p_j^{tea}\|_2^2. \quad (69)$$

若 `consistency_gate_only=true`, 一致性权重就是是否通过门控; 否则使用可靠性分数:

$$w_j^{cons} = \begin{cases} \mathbf{1}[j \in \Omega_t], & \text{gate-only,} \\ r_j^\eta, & \text{否则.} \end{cases} \quad (70)$$

于是主一致性项为

$$\mathcal{L}_{cons} = \frac{\sum_j w_j^{cons} v_j}{\sum_j w_j^{cons} + \varepsilon}. \quad (71)$$

仓库还把门控样本进一步分成 reliable / semi-reliable / unreliable 三段。对半可靠样本单独再加一项

$$\mathcal{L}_{semi} = \frac{\sum_{j \in \mathcal{S}_{semi}} r_j v_j}{\sum_{j \in \mathcal{S}_{semi}} r_j + \varepsilon}, \quad (72)$$

对不可靠样本加熵惩罚

$$\mathcal{L}_{ue} = \frac{1}{|\mathcal{S}_{unrel}|} \sum_{j \in \mathcal{S}_{unrel}} \frac{-\sum_k p_{jk}^{stu} \log p_{jk}^{stu}}{\log K}. \quad (73)$$

4.9 课程化总目标

伪标签、原型、一致性三项都可以带启动步和预热步。统一写成

$$\lambda_{aux}(t) = \begin{cases} 0, & t < t_0, \\ \lambda \cdot \min((t - t_0)/\Delta t, 1), & t \geq t_0. \end{cases} \quad (74)$$

于是当前仓库中的 RCTA 总目标准确写法是

$$\begin{aligned} \mathcal{L}_{RCTA} = & \mathcal{L}_{cls}^s + \lambda_{align}(t) \mathcal{L}_{align} + \lambda_{mcc} \mathcal{L}_{mcc} + \lambda_{pl}(t) \mathcal{L}_{pl} \\ & + \lambda_{proto}(t) \mathcal{L}_{proto} + \lambda_{cons}(t) \mathcal{L}_{cons} + \lambda_{semi} \mathcal{L}_{semi} + \lambda_{ue} \mathcal{L}_{ue}. \end{aligned} \quad (75)$$

这里最后两项 λ_{semi} 和 λ_{ue} 在当前实现中是固定系数, 不走 ramp 调度。

4.10 一步训练之后还会发生什么

当前步反向传播和优化器更新完成后，`after_optimizer_step()` 还会做三件事：

1. 用学生参数 EMA 更新教师；
2. 更新源原型记忆库与目标原型记忆库；
3. 步数 `step_num` 加 1。

这意味着 RCTA 不是“前向算完 loss 就结束”的静态模型，而是一个带状态演化的训练系统。

5 把七种方法放在一张图里理解

方法	核心对象	你应该抓住的数学本质
Source Only	$\mathcal{L}_{cls}^s$	只在源域学判别边界，不处理域偏移。
CORAL	协方差、均值	对齐特征二阶统计量。
DAN	RKHS 均值嵌入	用 MK-MMD 对齐边缘分布。
DANN	域判别器 + GRL	让特征对类别有用、对域无用。
CDAN	特征 $\times$ 类别概率	不只混域，还把类别条件一起送进对抗器。
DeepJDOT	运输计划 γ	显式匹配哪一个源样本更应该对应哪一个目标样本。
RCTA	可靠性、自训练、原型、课程调度	在强基线对齐之上，控制谁能参与目标域监督，以及何时参与、参与多深。

6 结语

如果把这套项目压缩成一句话，那就是：

共享 FCN 骨干负责学工业时序表征，基线方法各自定义“怎么跨域对齐”，而 RCTA 进一步回答“哪些目标样本值得信、怎样把类别结构稳定地注入训练”。

读懂本项目数学部分时，最值得反复盯住的不是某个单独公式，而是下面这条层次：

监督分类 $\rightarrow$ 全局对齐 $\rightarrow$ 条件结构 $\rightarrow$ 可靠伪监督 $\rightarrow$ 课程化稳定训练. (76)

这正是这个仓库从 Source Only 一路走到 RCTA 的主线。